Refatoração de Testes e Detecção de Test Smells

Disciplina: Teste de Software

Trabalho: Refatoração de Testes e Detecção de Test Smells

Aluno: Pedro Motta

---

1. Análise de Smells

Após analisar o arquivo `userService.smelly.test.js`, foram identificados os seguintes Test Smells:

1.1 Conditional Test Logic (Lógica Condicional no Teste)

Onde: Teste "deve desativar usuários se eles não forem administradores" (linhas 33-52)

```
for (const user of todosOsUsuarios) {
  const resultado = userService.deactivateUser(user.id);
  if (!user.isAdmin) {
    expect(resultado).toBe(true);
    const usuarioAtualizado = userService.getUserById(user.id);
    expect(usuarioAtualizado.status).toBe('inativo');
  } else {
    expect(resultado).toBe(false);
  }
}
```

Por que é um mau cheiro: O teste contém um loop `for` e uma estrutura `if/else`, o que torna impossível saber, à primeira vista, quais assertions estão sendo executadas e em qual ordem. Se o array de usuários mudar ou estiver vazio, os `expect` podem nunca ser executados, e o teste passa silenciosamente sem validar nada. Um teste deve ser linear e previsível — a presença de lógica condicional esconde o que realmente está sendo verificado.

Risco: Se o comportamento do `deactivateUser` mudar para um dos casos (admin ou comum), o teste pode não detectar a regressão, pois a lógica condicional mascara qual caminho está sendo exercitado.

1.2 Missing Assertion / Try-Catch sem Fail (Expect Condicional)

Onde: Teste "deve falhar ao criar usuário menor de idade" (linhas 66-75)

```
try {
  userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe('O usuário deve ser maior de idade.');
}
```

Por que é um mau cheiro: Se a função `createUser` parar de lançar a exceção (por exemplo, se alguém remover a validação de idade), o bloco `catch` nunca é executado, o `expect` nunca roda, e o teste **passa silenciosamente**. Isso cria uma falsa sensação de segurança — o teste diz que está tudo bem quando, na verdade, existe um bug grave.

Risco: Um bug real (remoção da validação de menor de idade) passaria completamente despercebido pela suíte de testes, que é exatamente o oposto do que um teste deve fazer.

1.3 Ignored Test / Teste Desabilitado

Onde: Teste "deve retornar uma lista vazia quando não há usuários" (linhas 77-79)

```
test.skip('deve retornar uma lista vazia quando não há usuários', () => {
  // TODO: Implementar este teste depois.
});
```

Por que é um mau cheiro: Um teste com `test.skip` é um teste que não está sendo executado. Ele representa um cenário de negócio que foi identificado como importante (comportamento do relatório sem usuários), mas que nunca foi validado. Com o tempo, testes desabilitados são esquecidos e a cobertura funcional fica incompleta.

Risco: O comportamento de "lista vazia" pode quebrar sem que ninguém perceba, já que nenhum teste o valida.

---

2. Processo de Refatoração

Teste escolhido: "deve desativar usuários se eles não forem administradores"

Este foi o teste mais problemático, pois combina **Eager Test** (testa dois comportamentos distintos no mesmo teste), **Conditional Test Logic** (usa `for` e `if/else`) e nomes pouco descritivos.

ANTES (arquivo smelly):

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

DEPOIS (arquivo clean):

```
test('deve desativar um usuário comum com sucesso', () => {
  // Arrange
  const usuario = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuario.id);

  // Assert
  expect(resultado).toBe(true);
  const atualizado = userService.getUserById(usuario.id);
  expect(atualizado.status).toBe('inativo');
});

test('não deve desativar um usuário administrador', () => {
  // Arrange
  const admin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(admin.id);

  // Assert
  expect(resultado).toBe(false);
  const atualizado = userService.getUserById(admin.id);
  expect(atualizado.status).toBe('ativo');
});

test('deve retornar false ao tentar desativar usuário inexistente', () => {
  // Act
  const resultado = userService.deactivateUser('id-inexistente');

  // Assert
  expect(resultado).toBe(false);
});
```

Decisões de refatoração:

1. **Separação de responsabilidades:** O teste único foi dividido em 3 testes independentes, cada um verificando um cenário específico (usuário comum, admin, inexistente). Isso segue o princípio de que cada teste deve validar um único comportamento.
2. **Remoção da lógica condicional:** O `for` e o `if/else` foram completamente eliminados. Cada teste é linear, seguindo o padrão AAA (Arrange, Act, Assert), o que torna o fluxo previsível e fácil de entender.
3. **Nomes descritivos:** Cada teste tem um nome que descreve exatamente o cenário e o resultado esperado, facilitando a identificação de falhas.
4. **Teste adicional:** Foi adicionado um teste para o cenário de usuário inexistente, que não era coberto no original.

---

3. Relatório da Ferramenta (ESLint)

Após configurar o ESLint com o plugin `eslint-plugin-jest` e as regras `jest/no-conditional-expect`, `jest/no-disabled-tests` e `jest/no-identical-title`, o seguinte resultado foi obtido ao executar `npx eslint .` no arquivo original:

```
userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped           jest/no-disabled-tests
  77:3  warning  Test has no assertions                jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Análise do resultado:

- **4 erros `jest/no-conditional-expect`:** A ferramenta detectou automaticamente que existem chamadas `expect` dentro de estruturas condicionais (`if/else` nas linhas 44, 46, 49 e `try/catch` na linha 73). Isso corresponde exatamente aos smells de Conditional Test Logic e Missing Assertion que identificamos manualmente.
- **1 warning `jest/no-disabled-tests`:** A ferramenta identificou o `test.skip` na linha 77 como um teste desabilitado, correspondendo ao smell de Ignored Test.
- **1 warning `jest/expect-expect`:** A ferramenta apontou que o teste skipado não possui nenhuma assertion, reforçando que é um teste vazio e inútil.

Após a refatoração, o ESLint foi executado no arquivo `userService.clean.test.js` e retornou **0 erros e 0 warnings**, confirmando que todos os smells detectáveis pela ferramenta foram corrigidos.

---

4. Conclusão

Este trabalho demonstrou que testes que "passam" não são necessariamente testes de qualidade. A suíte original tinha 100% de aprovação, mas escondia problemas graves: assertions que podiam nunca ser executadas, lógica condicional que mascarava falhas e testes desabilitados que deixavam cenários sem cobertura.

A combinação de **análise manual** e **ferramentas de análise estática** como o ESLint se mostrou eficaz para detectar esses problemas. A análise manual permite identificar smells conceituais como Eager Test e Teste Frágil, enquanto o ESLint automatiza a detecção de padrões problemáticos como expects condicionais e testes desabilitados.

A aplicação do padrão **Arrange, Act, Assert (AAA)** trouxe clareza e previsibilidade aos testes. Cada teste passou a ter um único propósito, nomes descritivos e um fluxo linear, tornando a suíte mais fácil de manter e mais confiável na detecção de bugs.

A lição principal é que a qualidade do código de teste merece a mesma atenção que a do código de produção. Ferramentas de linting integradas ao fluxo de desenvolvimento ajudam a manter essa qualidade de forma contínua e sustentável.